

```

1 #=====
2 # MIT 6.046J 代码精读 • PS9 - 背包近似 FPTAS
3 # 源文件: knapsack.py (来自 GitHub mit-6046)
4 #=====
5
6 """PS9 Problem 9-1: Knapsack approximation algorithms.
7
8 Items a_i with integer size s_i and value v_i, capacity B (s_i <= B). Maximize
9 total value of a subset with total size <= B.
10
11 alg1_density_greedy : (a) greedy by density — NO constant ratio.
12 alg2_two_approx     : (b) greedy-or-single — provably a 2-approximation.
13 alg3_dp_exact       : (c) exact DP over achievable values, O(n^2 V).
14 alg4_fptas          : (d) FPTAS via value scaling + Alg3, (1-eps)-approx in
15                     time poly(n, 1/eps).
16
17 An independent brute-force exact solver certifies Alg3 and bounds the others.
18 """
19 from __future__ import annotations
20
21 import itertools
22 import math
23 from typing import List, Tuple
24
25 Item = Tuple[int, int] # (size, value)
26
27 def _total(items, subset):
28     s = sum(items[i][0] for i in subset)
29     v = sum(items[i][1] for i in subset)
30     return s, v
31
32
33
34 # -----
35 # (a) density greedy
36 # -----
37 def alg1_density_greedy(items: List[Item], B: int) -> Tuple[int, List[int]]:
38     order = sorted(range(len(items)), key=lambda i: items[i][1] / items[i][0], reverse=True)
39     size = 0
40     chosen = []
41     for i in order:
42         if size + items[i][0] <= B:
43             size += items[i][0]
44             chosen.append(i)
45     return sum(items[i][1] for i in chosen), sorted(chosen)
46
47
48 def alg1_counterexample():
49     """No constant ratio: item0 tiny/high-density but low value, item1 fills B
50     with huge value. Greedy grabs item0 first, then item1 doesn't fit."""
51     # densities: item0 = 2/1 = 2 ; item1 = k/k = 1 -> greedy takes item0 only.
52     k = 1000
53     items = [(1, 2), (k, k)] # sizes, values
54     B = k
55     val, _ = alg1_density_greedy(items, B) # takes item0 (value 2)
56     opt = k # take item1
57     # ratio opt/greedy = k/2 -> unbounded as k grows
58     return {"items": items, "B": B, "greedy_value": val, "opt_value": opt,
59           "ratio": opt / val}
60
61
62 # -----
63 # (b) 2-approximation
64 # -----
65 def alg2_two_approx(items: List[Item], B: int) -> Tuple[int, List[int]]:
66     n = len(items)
67     if sum(s for s, _ in items) <= B:
68         return sum(v for _, v in items), list(range(n))
69     order = sorted(range(n), key=lambda i: items[i][1] / items[i][0], reverse=True)
70     size = 0
71     prefix = []
72     i_break = None
73     for pos, i in enumerate(order):
74         if size + items[i][0] <= B:
75             size += items[i][0]
76             prefix.append(i)
77         else:
78             i_break = i
79             break
80     prefix_val = sum(items[i][1] for i in prefix)
81     break_val = items[i_break][1] if i_break is not None else 0
82     if break_val > prefix_val:
83         return break_val, [i_break]
84     return prefix_val, sorted(prefix)
85

```

```

86
87 # -----
88 # (c) exact DP over values,  $O(n^2 V)$  using  $S_{i,v}$  = min size for value exactly v
89 # -----
90 def alg3_dp_exact(items: List[Item], B: int) -> Tuple[int, List[int]]:
91     n = len(items)
92     V = max((v for _, v in items), default=0)
93     Vmax = n * V
94     INF = float("inf")
95     # S[v] = min total size achieving total value exactly v using items seen so far
96     S = [INF] * (Vmax + 1)
97     S[0] = 0
98     choice = [[False] * (Vmax + 1) for _ in range(n)] # choice[i][v]: item i used to reach v
99     parent = [[-1] * (Vmax + 1) for _ in range(n + 1)]
100    # DP with reconstruction: process items, track predecessor value
101    prev = S[:]
102    take = [[None] * (Vmax + 1) for _ in range(n)]
103    for i in range(n):
104        s_i, v_i = items[i]
105        cur = prev[:]
106        for v in range(Vmax, v_i - 1, -1):
107            if prev[v - v_i] + s_i < cur[v]:
108                cur[v] = prev[v - v_i] + s_i
109                take[i][v] = True
110        prev = cur
111    best_v = 0
112    for v in range(Vmax + 1):
113        if prev[v] <= B:
114            best_v = v
115    # reconstruct
116    chosen = []
117    v = best_v
118    for i in range(n - 1, -1, -1):
119        if v >= items[i][1] and take[i][v]:
120            # ambiguous reconstruction; recompute forward-safe below
121            pass
122    chosen = _reconstruct_knapsack(items, B, best_v)
123    return best_v, chosen
124
125
126 def _reconstruct_knapsack(items, B, target_value):
127     """Recompute a min-size subset achieving exactly target_value (<=B)."""
128     n = len(items)
129     INF = float("inf")
130     Vmax = target_value
131     # dp[i][v] = min size using first i items to reach value exactly v
132     dp = [[INF] * (Vmax + 1) for _ in range(n + 1)]
133     dp[0][0] = 0
134     for i in range(1, n + 1):
135         s_i, v_i = items[i - 1]
136         for v in range(Vmax + 1):
137             dp[i][v] = dp[i - 1][v]
138             if v >= v_i and dp[i - 1][v - v_i] + s_i < dp[i][v]:
139                 dp[i][v] = dp[i - 1][v - v_i] + s_i
140     chosen = []
141     v = target_value
142     for i in range(n, 0, -1):
143         s_i, v_i = items[i - 1]
144         if v >= v_i and dp[i - 1][v - v_i] + s_i == dp[i][v] and dp[i][v] != INF:
145             chosen.append(i - 1)
146             v -= v_i
147     return sorted(chosen)
148
149
150 # -----
151 # (d) FPTAS
152 # -----
153 def alg4_fptas(items: List[Item], B: int, eps: float) -> Tuple[int, List[int]]:
154     n = len(items)
155     V = max((v for _, v in items), default=0)
156     if V == 0:
157         return 0, []
158     scale = (eps * V) / n
159     if scale <= 0:
160         return alg3_dp_exact(items, B)
161     scaled = [(s, int(v / scale)) for (s, v) in items] # floor(v_i / scale)
162     _, chosen = alg3_dp_exact(scaled, B)
163     # report the TRUE value of the chosen set
164     true_val = sum(items[i][1] for i in chosen)
165     return true_val, chosen
166
167
168 # -----
169 # brute-force exact oracle
170 # -----
171 def brute_exact(items: List[Item], B: int) -> Tuple[int, List[int]]:

```

```
172     n = len(items)
173     best_v = 0
174     best = []
175     for r in range(n + 1):
176         for combo in itertools.combinations(range(n), r):
177             s = sum(items[i][0] for i in combo)
178             if s <= B:
179                 v = sum(items[i][1] for i in combo)
180                 if v > best_v:
181                     best_v, best = v, list(combo)
182     return best_v, best
183
```